



Unit-1

Short Question and Answers

1) Define Number System. List out different types of number systems?

Ans: A number system is a method of representing numbers using a set of digits or symbols according to certain rules. It is used to store, process, and communicate data in digital devices.

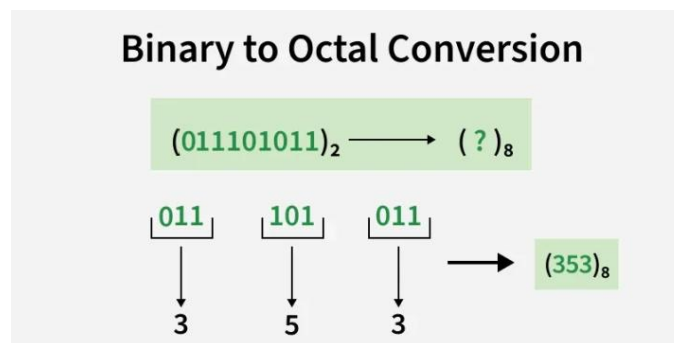
➤ In digital electronics, there are four types of digital number systems are mainly used:

- Binary Number System
- Octal Number System
- Decimal Number System
- Hexadecimal Number System.

2) Convert $(011101011)_2$ into an octal number.

Ans: Step 1: Divide the binary number into groups of three digits starting from right to left i.e. from LSB to MSB.

Step 2: Convert these groups into equivalent octal digits.



3) What is 2's Complement? Give one example.

Ans: To find the 2's complement, calculate the 1's complement first, then add 1 to the least significant bit (the far right). This is the standard method computers use to represent signed integers.



2's Complement

Binary Number = 101011

Step 1: 1's Complement – 010100

Step 1: 2's Complement – 010100
 +1 [Add 1]
 010101

4) Define De Morgan's Law.

Ans: This law states that the complement of the AND operation of two variables is equal to the OR operation of their complements, and the complement of the OR operation of two variables is equal to the AND operation of their complements. Mathematically, the laws are expressed as:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$
$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

5) Convert (B2E)₁₆ to Octal number.

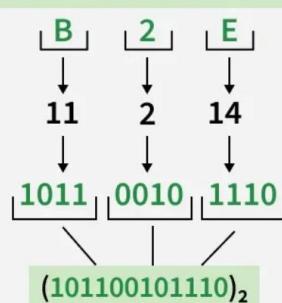
Ans: Step 1: To convert Hexadecimal to Octal, first we need to convert the Hexadecimal number to Binary.

Step 2: Now to convert the binary number to an Octal number, divide the binary digits into groups of three digits starting from right to left i.e. from LSB to MSB.

Hexadecimal to octal Conversion

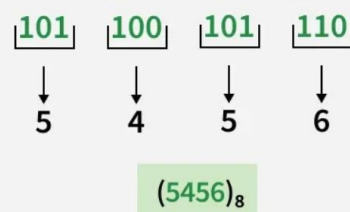
Hexadecimal to Binary

(B2E)₁₆ → (?)₈



Binary to Octal

(101100101110)₂



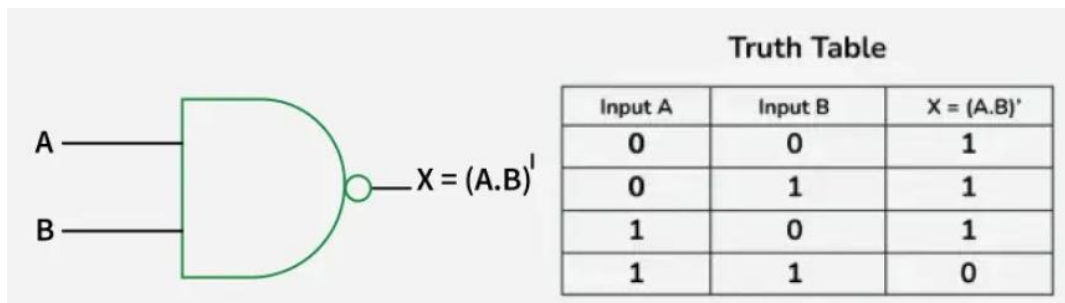


6) Explain about NAND gate With symbol and truth table.

Ans: The NAND gate is the combination of two basic logic gates AND gate and the NOT gate connected in series.

- The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs the inverted operation of AND gate.
- The Boolean Expression of NAND Gate is as follows:

$$X = \overline{A.B} \text{ or } (A.B)'$$

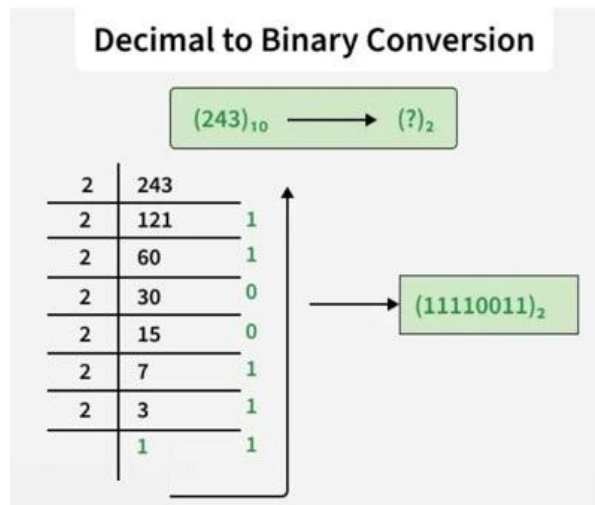


Long Question and Answers

1) Convert $(243)_{10}$ into following number system:

- a) Binary number b) Octal number c) Hexadecimal number.

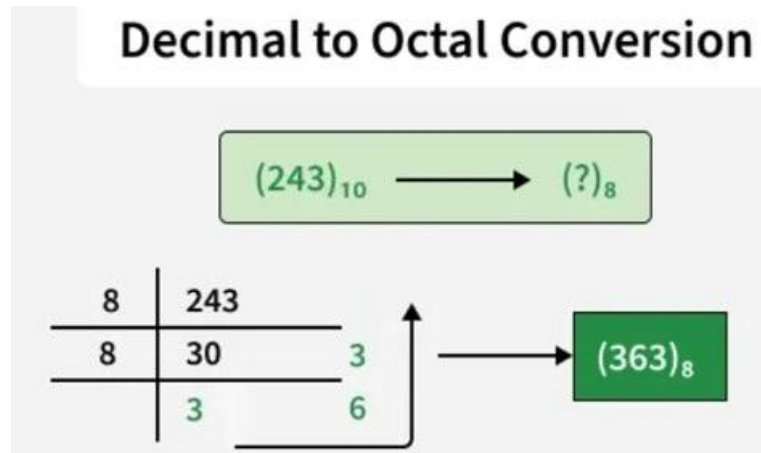
Ans: a) binary number: Decimal numbers are represented in base 10, but the Binary numbers are of base 2. Hence, to convert the decimal number to a Binary number, the base of that number is to be changed.



$$(243)_{10} = (11110011)_2$$

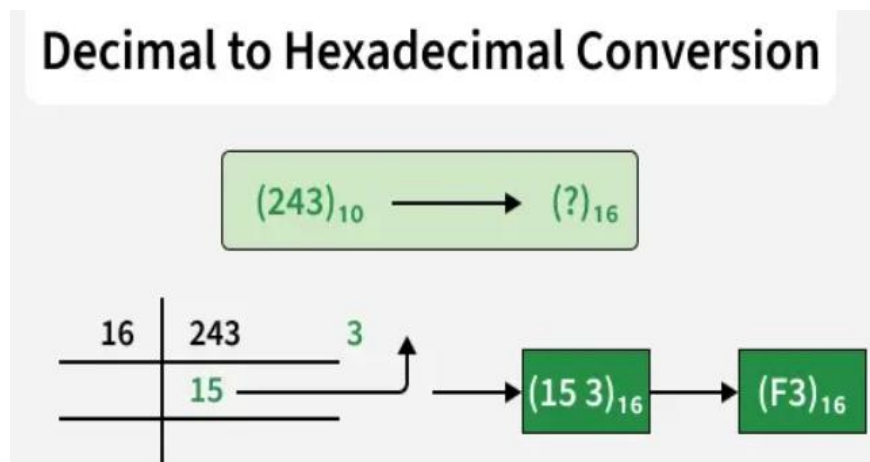


b) Octal number : Decimal numbers are represented in base 10, but the Octal numbers are of base 8. Hence, to convert the decimal number to an Octal number, the base of that number is to be changed.



$$(243)_{10} = (368)_8$$

c) Hexadecimal number: Decimal numbers are represented in base 10, but the Hexadecimal numbers are of base 16. Hence, to convert the decimal number to a Hexadecimal number, the base of that number is to be changed.



$$(243)_{10} = (F3)_{16}$$



2) **Convert $(247)_8$ into following number system:**

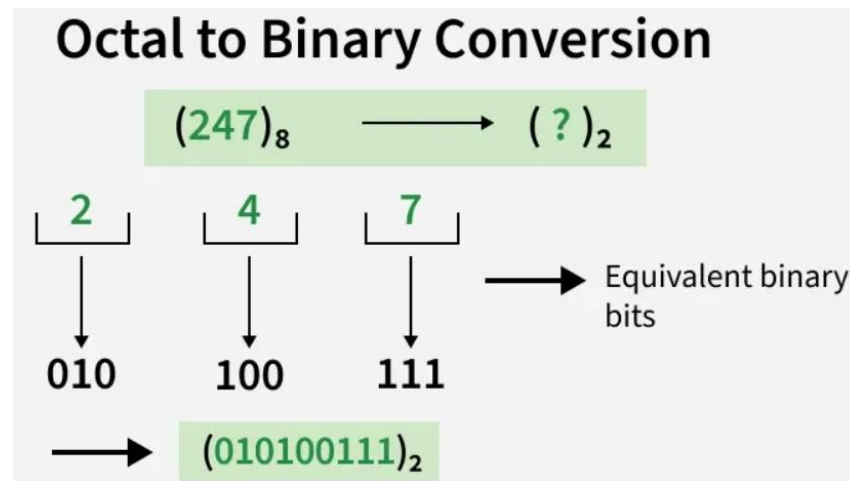
- a) Binary number b) Decimal number c) Hexadecimal number.

Ans: a) Binary number:

Step 1: Write each digit of the octal number separately.

Step 2: Convert each digit into an equivalent group of three binary digits.

Step 3: Combine these groups to form the whole binary number.



b) Decimal number:

Step 1: Multiply each digit of the Octal number with the place value of that digit, starting from right to left i.e. from LSB to MSB.

Step 2: Add the result of this multiplication and the decimal number will be formed.

$$\begin{aligned}(247)_8 &= 2 \times 8^2 + 4 \times 8^1 + 7 \times 8^0 \\ &= 2 \times 64 + 4 \times 8 + 7 \times 1 \\ &= 128 + 32 + 7 \\ &= 167\end{aligned}$$

$$(247)_8 = (167)_{10}$$



c) Hexadecimal number:

Step 1: To convert Octal number into Hexadecimal, first convert that octal number into binary number.

Replace each octal digit with its 3-bit binary equivalent:

- $2 \rightarrow 010$
- $4 \rightarrow 100$
- $7 \rightarrow 111$

$$(247)_8 = (010100111)_2$$

Step 2: After binary conversion, convert that binary number into hexadecimal number.

$$010100111 \rightarrow 0000 \ 1010 \ 0111$$

- $0111 \rightarrow 7$
- $1010 \rightarrow 10 \rightarrow A$
- $0000 \rightarrow 0$

$$(247)_8 = (0A7)_{16}$$

3) What is Boolean Algebra? Give any five properties of Boolean Algebra.

Ans: Boolean algebra is a branch of mathematics that provides a set of operations and rules to manipulate and process binary variables.

- Boolean algebra is based on the binary number system.
- The operations in Boolean algebra are based on the three fundamental logical operations namely, AND, OR, and NOT.

i) **Identity Law :** The identity law state that in boolean algebra we have such variables that on operating with AND and OR operation we get the same result, i.e.

- $A + 0 = A$
- $A \cdot 1 = A$

ii) **Complement law:** In this Law if a complement is added to a variable it gives 1, if a variable is multiplied with its complement it results in '0', i.e.,

- $A + A' = 1$
- $A \cdot A' = 0$



- iii) **Idempotent law:** A variable remains unchanged when it is OR or AND with itself, i.e.,

$$A + A = A$$

$$A.A = A$$

- iv) **Commutative Law:** This law states that operating boolean variables A and B is similar to operating boolean variables B and A. That is,

$$A . B = B . A$$

$$A + B = B + A$$

- v) **Demorgans Law:** This law state that the complement of the AND operation of two variables is equal to the OR operation of their complements, and the complement of the OR operation of two variables is equal to the AND operation of their complements. Mathematically, the laws are expressed as:

$$\overline{A.B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} . \overline{B}$$

- vi) **Associative Law:** This law state that the order of performing Boolean operator is illogical as their result is always the same. This can be understood as,

$$(A . B) . C = A . (B . C)$$

$$(A + B) + C = A + (B + C)$$

- vii) **Distributive Law:** The expression for Distributive law is given as

$$A . (B + C) = (A . B) + (A . C)$$

$$A + (B . C) = (A + B) (A + C)$$

- viii) **Inversion Law:** Inversion law is the unique law of Boolean algebra this law states that, the complement of the complement of any number is the number itself.

$$(A')' = A$$

- ix) **Dominance law:** Inversion law is the unique law of Boolean algebra this law



4) a) Express the Boolean function $F = (x + y z)$ as a sum of minterms.

b) Express the Boolean function $F = (x + y z)$ as a product of maxterms.

Ans: a) This function has three variables: x , y , and z . All terms must have these three variables. Thus, we need to expand the first term by AND operation it with $(y + y')(z + z')$, and we expand the second term with $(x + x')$ to get,

$$\begin{aligned} F &= x + y z \\ &= x (y + y') (z + z') + (x + x') y z \\ &= x y z + x y z' + x y' z + x y' z' + x y z + x' y z \\ &= x' y z + x y' z' + x y' z + x y z' + x y z \\ &= m_3 + m_4 + m_5 + m_6 + m_7 \\ &= \Sigma(3, 4, 5, 6, 7) \end{aligned}$$

x	y	z	Minterms
0	0	0	$m_0 = x' y' z'$
0	0	1	$m_1 = x' y' z$
0	1	0	$m_2 = x' y z'$
0	1	1	$m_3 = x' y z$
1	0	0	$m_4 = x y' z'$
1	0	1	$m_5 = x y' z$
1	1	0	$m_6 = x y z'$
1	1	1	$m_7 = x y z$

b) First, we need to convert the function into the product-of-OR terms by using the distributive law as follows:

$$\begin{aligned} F &= x + y z \\ &= (x + y) (x + z) \quad (\text{distributive law}) \\ &= (x + y + z z') (x + y y' + z) \quad (\text{expand 1st term with } z z', \text{ and 2nd term with } y y') \\ &= (x + y + z) (x + y + z') (\cancel{x + y + z}) (x + y' + z) \\ &= M_0 \cdot M_1 \cdot M_2 \\ &= \Pi(0, 1, 2). \end{aligned}$$



x	y	z	Maxterms
0	0	0	$M_0 = x + y + z$
0	0	1	$M_1 = x + y + z'$
0	1	0	$M_2 = x + y' + z$
0	1	1	$M_3 = x + y' + z'$
1	0	0	$M_4 = x' + y + z$
1	0	1	$M_5 = x' + y + z'$
1	1	0	$M_6 = x' + y' + z$
1	1	1	$M_7 = x' + y' + z'$

5) What are Logic gates? Briefly explain about Basic logic gates with symbols and truth tables.

Ans: Logic gates are the fundamental building blocks in digital electronics.

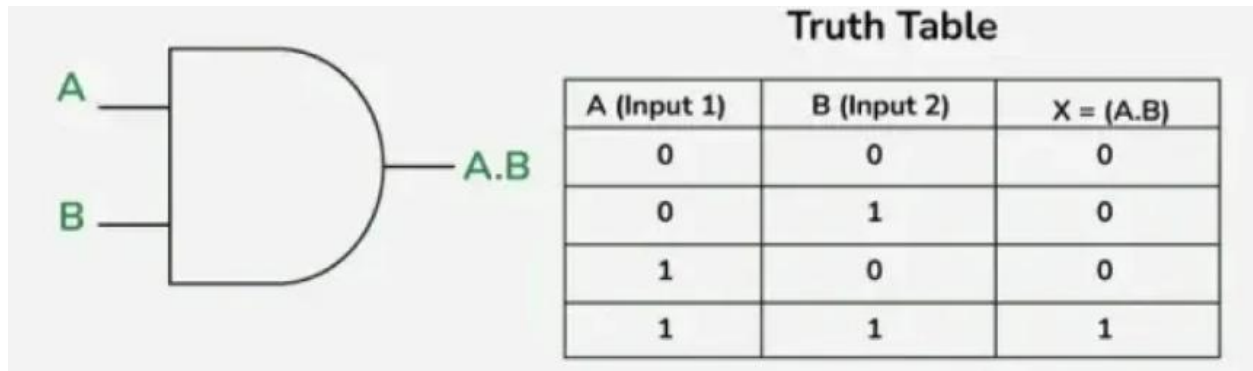
- Used to perform logical operations based on the inputs provided to it and gives a logical output that can be either high(1) or low(0).
- Basic Logic Gates :- AND Gate, OR Gate, NOT Gate

1.AND GATE:

- An AND gate is used to perform logical Multiplication of binary input.
- The Output state of the AND gate will be high (1) if both the input is high (1), else the output state will be low(0) if any of the input is low (0).
- The Boolean Expression or logic for the AND gate is the logical multiplication of inputs denoted by a full stop or single dot as :

$$X = A.B$$

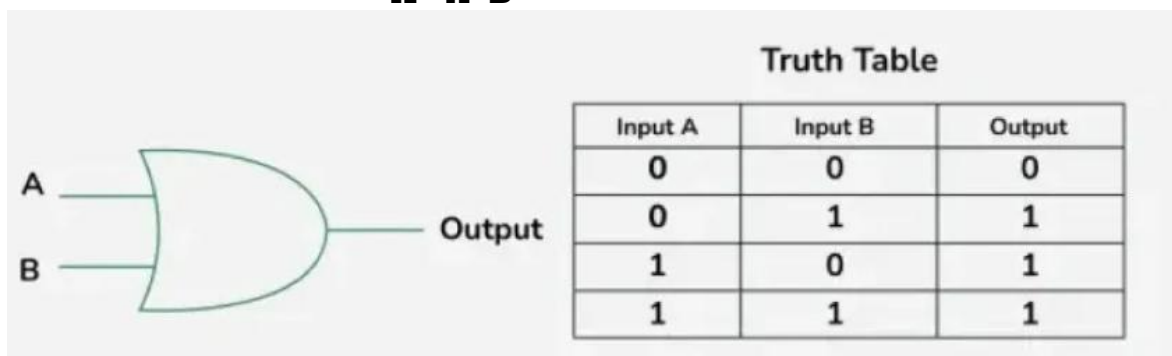
- The value of X will be True when both the inputs will be True.



2.OR GATE:

- The OR gate is used to perform logical addition operation of binary input.
- The output state of OR gate will be high(1), if any of the input state is high(1), else output state will be low (0).
- The Boolean Expression for the OR gate is the logical addition of inputs denoted by plus sign (+) as

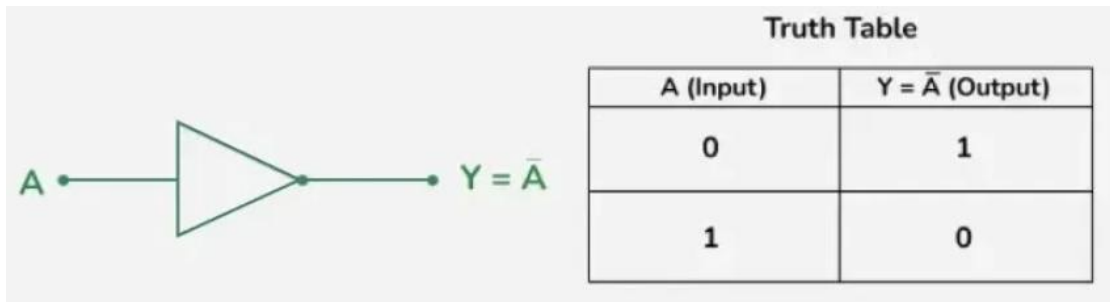
$$X = A + B$$



3.NOT GATE:

- The NOT gate is one of the basic Logic Gate having only a single input and a single output. It is also known as inverter or inverting buffer.
- When the input signal is "low" the output signal is "high" and vice-versa.
- The Boolean expression of NOT Gate is as follows:

$$Y = \bar{A} \quad \text{or} \quad Y = A'$$



6) Explain briefly about Universal logic gates and Derived logic gates with symbols and truth tables.

Ans:

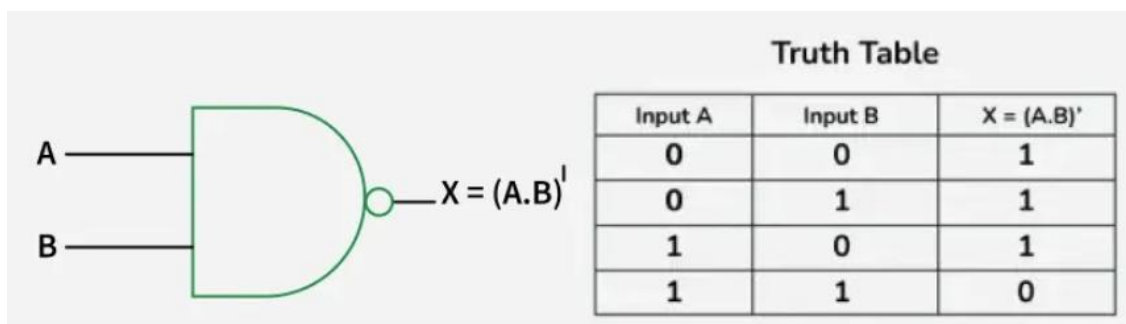
- Universal Logic Gates :- NAND Gate and NOR Gate
- Derived Logic Gates :- XOR Gate and XNOR Gate

Universal Logic Gates:

NAND GATE:

- The NAND gate is the combination of two basic logic gates AND gate and the NOT gate connected in series. It takes two or more inputs and gives only one output.
- The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs the inverted operation of AND gate.
- The Boolean Expression of NAND Gate is as follows:

$$X = \overline{A.B} \quad \text{or} \quad X = (A.B)'$$

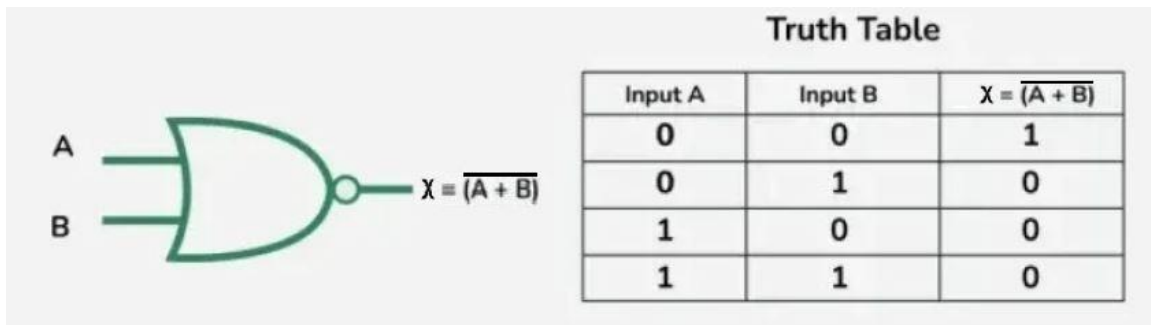




NOR GATE:

- The output state of the NOR gate will be high (1) when all the inputs are low (0).
- NOR gate returns the complement result of the OR gate. It is basically a combination of two basic logic gates i.e., OR gate and NOT gate.
- The Boolean expression of NOR gate is as follows:

$$X = \overline{A+B} \quad \text{or} \quad X = (A+B)'$$

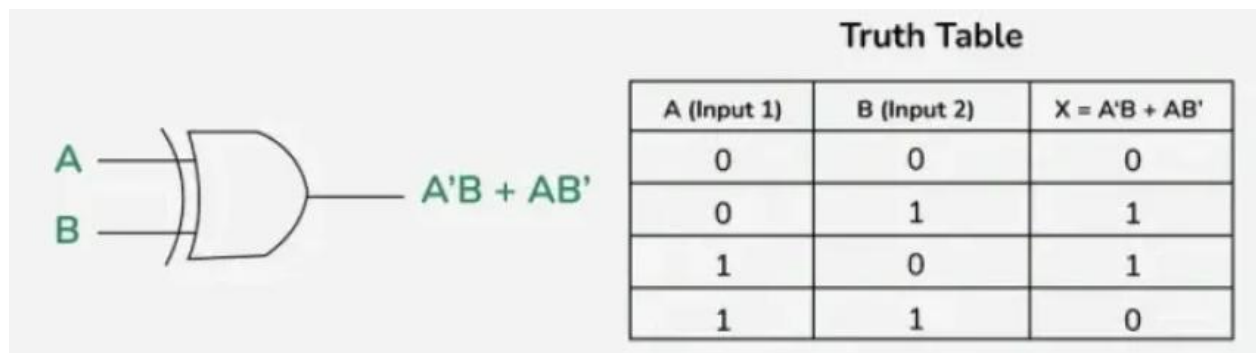


Derived Logic Gates:

XOR GATE:

- It is also referred to as **Exclusive OR gate or Ex-OR gate.** it is used extensively in arithmetic logic circuits., logic comparators and error detection circuits.
- The output of the XOR gate is high (1) only when its two inputs are dissimilar i.e., if one of them is low (0) then other one will be high (1).
- The Boolean expression of XOR Gate is as follows:

$$X = A'B + AB'$$

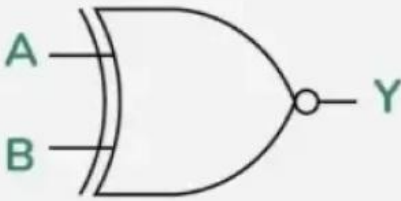




XNOR GATE:

- The XNOR is the combination of XOR gate and NOT gate. The output of the XNOR gate is high(1) when both the inputs are high (1) or low(0). In other words, the output of the XNOR gate is high(1) when both the inputs are the same.
- The following is the Boolean expression of the XNOR gate,

$$Y = (A \oplus B)' \quad \text{or} \quad \overline{(A \oplus B)}$$

		Truth Table		
	Input A	Input B	Output	
	0	0	1	
	0	1	0	
	1	0	0	
	1	1	1	



Unit-2

Short Question and Answers

1) What is the K-Map? what are the steps to solve an expression using K-Map?

Ans: A Karnaugh map (K-map) is a visual method used to simplify Boolean expressions and truth tables.

Steps to Solve Expression using K-map:

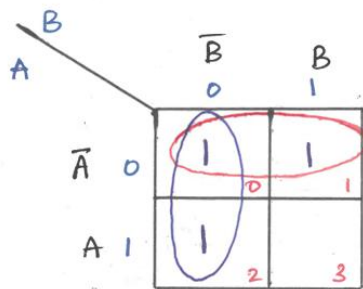
1. Select the K-map according to the number of variables.
2. Identify minterms or maxterms as given in the problem.
3. For SOP put 1's in blocks of K-map respective to the minterms (0's elsewhere).
4. For POS put 0's in blocks of K-map respective to the max terms (1's elsewhere).
5. Make rectangular groups containing total terms in power of two like 2,4,8 ..(except 1) and try to cover as many elements as you can in one group.
6. From the groups made in step 5 find the product terms and sum them up for SOP form.

2) Simplify the Boolean function $F(A,B) = \sum m(0,1,2)$.

Ans:

Solution: Given that,

$$F(A,B) = \sum m(0,1,2)$$



here two groups are formed, from these two groups,

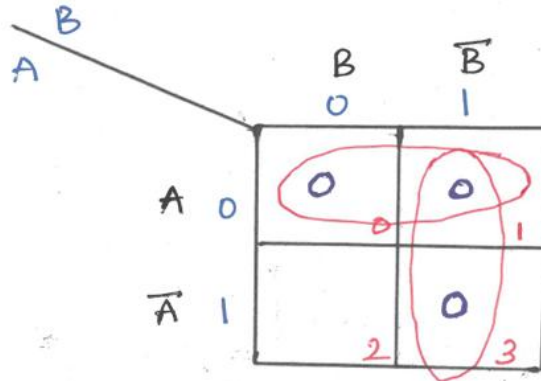
$$= \bar{A} + \bar{B}$$



3) Simplify the Boolean function $F(A,B) = \prod M(0,1,3)$.

Solution: Given that,

$$F(A,B) = \prod M(0,1,3)$$

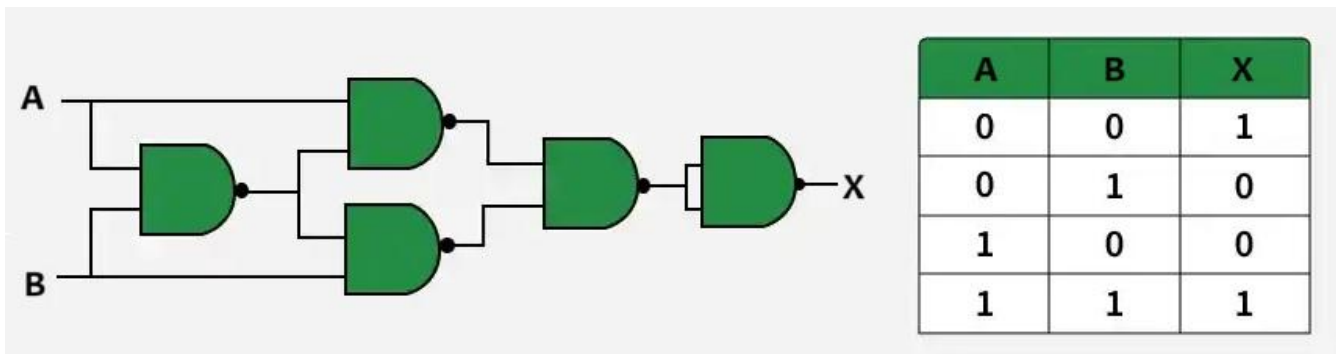


here two groups are formed, so from that two groups.

$$= (A)(\bar{B})$$

4) Describe the implementation of XNOR gate using NAND gate.

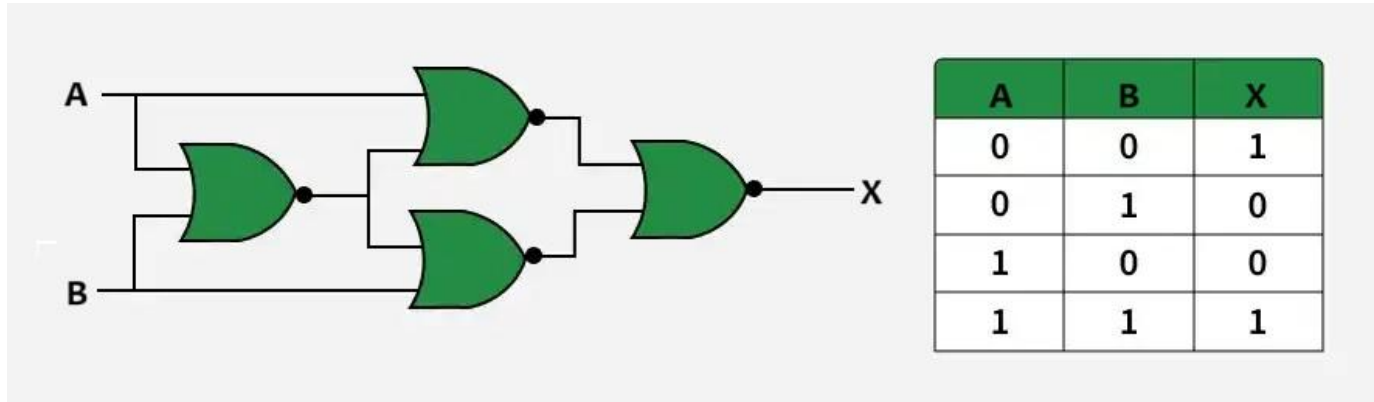
- The XNOR gate is implemented by five NAND gates.
- The first four NAND gates generate the XOR function, while the fifth NAND gate acts as a NOT gate to invert the XOR output and produce the XNOR output.
- Implementation of XNOR gate using five NAND gate as shown in the picture below:





5) Describe the implementation of XNOR gate using NOR gate.

Ans: The XNOR gate can be implemented using four NOR gates. The NOR gates generate intermediate outputs that are combined to produce the XNOR output.



6) Simplify the Boolean Expression $F = A'B'C + A'BC + AB'$

Solution: $F = A'B'C + A'BC + AB'$

$$= A'C(B' + B) + AB' \quad [\text{by complement law, } B + B' = 1]$$

$$= A'C + AB'$$



Long Question and Answers

1) Simplify the Boolean functions

a) $F(A,B) = \sum m(1,2)$.

b) $F(A,B) = \prod M(0,3)$.

a)

Solution:- Given that, $F(A,B) = \sum m(1,2)$

		B	
		$\bar{B}$ 0	B 1
A	$\bar{A}$ 0		1 ₁
	A 1	1 ₂	

here, no groups are formed, so we are taking individual expressions

$$= \bar{A}B + A\bar{B}$$



b)

Solution ; Given that,

$$F(A,B) = \Pi M(0,3)$$

		B	
		0	1
A	0	0	
	1		0

here no groups are formed, So we are taking individual Expressions.

$$= (A+B)(\bar{A}+\bar{B})$$



2) **Simplify the Boolean functions**

a) $F(A,B,C) = \sum m(1,2,3,7)$.

b) $F(A,B,C) = \prod M(0,1,3,5)$.

a)

Solution: Given that, $F(A,B,C) = \sum m(1,2,3,7)$

A \ BC				
	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	BC 11	$B\overline{C}$ 10
$\overline{A}$ 0	0	1	1	1
A 1	0	0	1	0

From these three groups, Final Expression is

$$= \overline{A}C + \overline{A}B + BC$$

b)

Solution:

A \ BC				
	BC 00	$B\overline{C}$ 01	$\overline{B}\overline{C}$ 11	$\overline{B}C$ 10
A 0	0	0	0	0
$\overline{A}$ 1	0	0	0	0

$$\Rightarrow (A+B)(A+\overline{C})(B+\overline{C})$$



3) Simplify the following Boolean Expressions

a) $F(A,B,C) = AB + (AC)' + AB'C(AB+C)$

b) $F(A,B,C) = (A+B)(A+C)$

a) Solution: $F = AB + (AC)' + AB'C(AB + C)$
 $= AB + A' + C' + AB'C.AB + AB'C.C$
 $= AB + A' + C' + 0 + AB'C (B.B' = 0 \text{ and } C.C = C)$
 $= ABC + ABC' + A' + C' + AB'C \quad [AB = AB(C + C') = ABC + ABC']$
 $= AC(B + B') + C'(AB + 1) + A'$
 $= AC + C' + A' (B + B' = 1 \text{ and } AB + 1 = 1)$
 $= AC + (AC)'$
 $= 1$

b) Solution: $F(A,B,C) = (A+B)(A+C)$
 $= A.A + A.C + B.A + B.C$
 $= A + A.C + B.A + B.C \quad [\text{Applying Idempotent Law}]$
 $= A(1+C) + B.A + B.C \quad [\text{Applying dominance Law}]$
 $= A + B.A + B.C$
 $= A(1+B) + B.C$
 $= A + B.C$



4) Simplify the following Boolean functions

a) $F(A,B,C) = \sum m(1,3,4,7) + \sum d(2,5)$

b) $F(A,B,C) = \prod M(0,1,3,6) + \prod M(5,7)$

a)

Solution:

A \ BC				
	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 11	BC 10
$\overline{A}$ 0		1	1	X
A 1	1	X	1	

From these three Groups, final Expression is
 $= \overline{A}B + C$

b)

Solution:

A \ BC				
	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 11	BC 10
A 0	0	0	0	
$\overline{A}$ 1		X	X	0

From these groups, final Expression is
 $= (A+B)(\overline{C})(\overline{A}+\overline{B})$

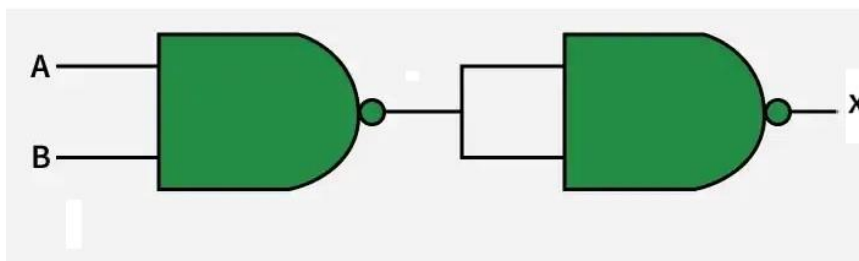


5) Describe the implementation of the following gates using NAND gate

a) AND gate b) OR gate c) NOT gate

Ans: a) Implementation of AND Gate using NAND Gates:

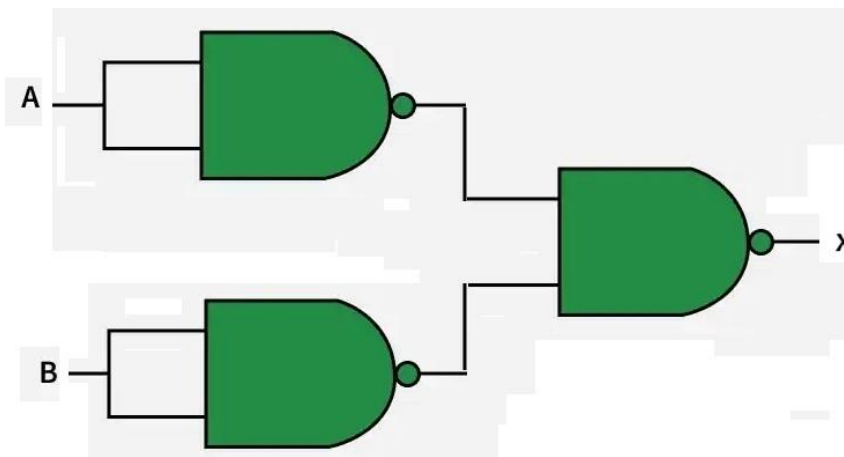
The AND gate is implemented by using two NAND gates where the output is HIGH only when both inputs are HIGH. The first NAND gate performs a standard NAND functions, the second NAND gate has both inputs tied to the output of the initial NAND gate as shown below.



Input		Output
A	B	$X = A.B$
0	0	0
0	1	0
1	0	0
1	1	1

b) Implementation of OR Gate using NAND Gates:

- It is implemented by using three NAND gates. The inputs are first inverted using two NAND gates having their inputs probed in parallel and then the inverted outputs are connected with the input of third NAND gate. The last NAND gate effectively performs the inverted signals to give the required OR gate function.
- The OR gate can be implemented using the NAND gate as below:

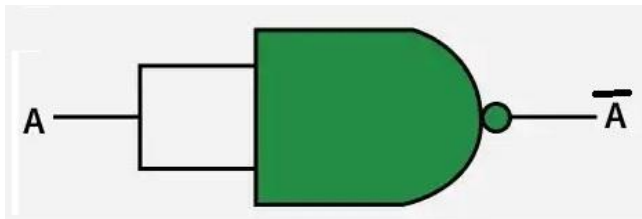


Input		Output
A	B	$X = A+B$
0	0	0
0	1	1
1	0	1
1	1	1



c) Implementation of NOT Gate using NAND Gates:

- A single NAND gate with 'a' and 'b' inputs connected respond to a NOT gate.
- Implementation of NOT gate using a single NAND gate as shown in the picture below:



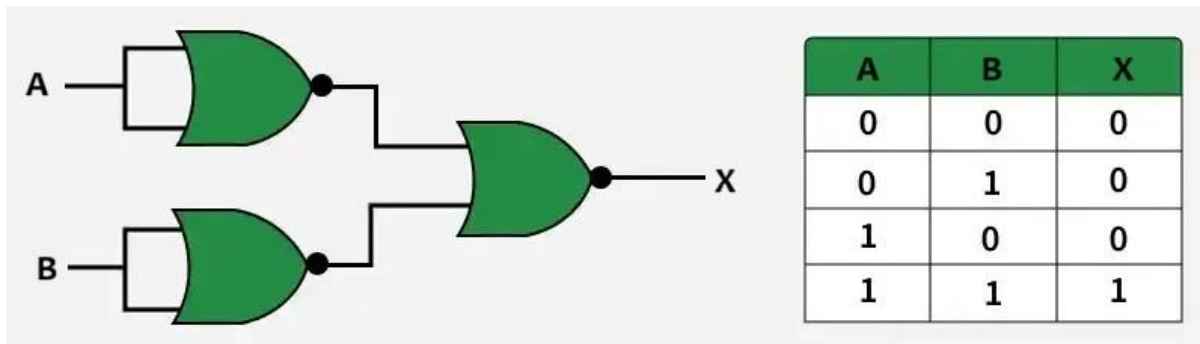
A (Input)	$\bar{A}$ (Output)
0	1
1	0

6) Describe the implementation of the following gates using NOR gate

a) AND gate b) OR gate c) NOT gate

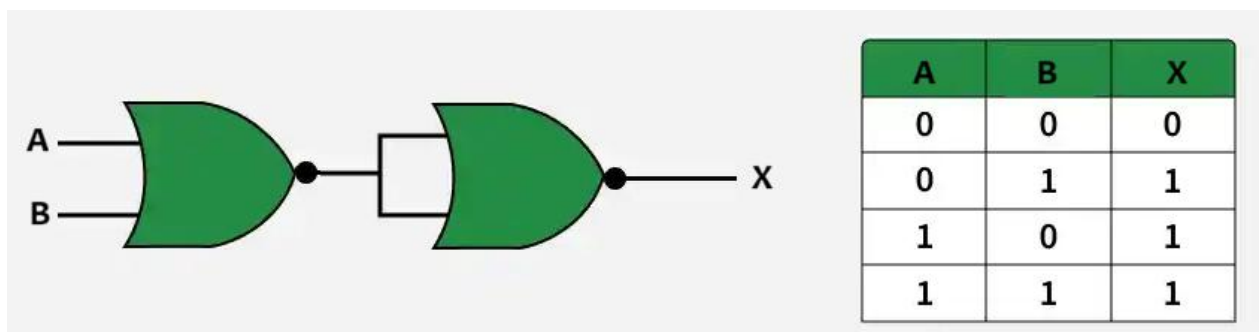
a) Implementation of AND gate using NOR Gates:

- AND gate is implemented by using three NOR gates.
- Implementation of AND gate using only NOR gates as shown below



b) Implementation of OR gate using NOR Gates:

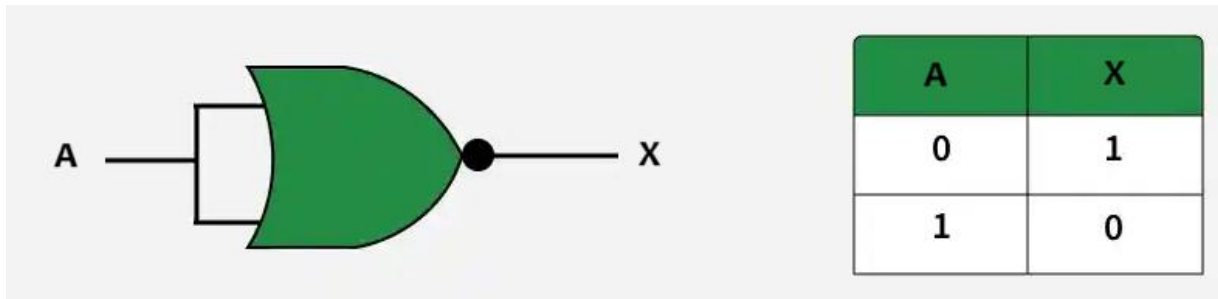
- OR gate is implemented by using two NOR gates.
- Implementation of OR gate using only NOR gates as shown below:





c) Implementation of NOT gate Using NOR Gates:

- NOT gate is implemented by using one NOR gate.
- Implementation of NOT gate using a single NOR gate as shown in the picture below:



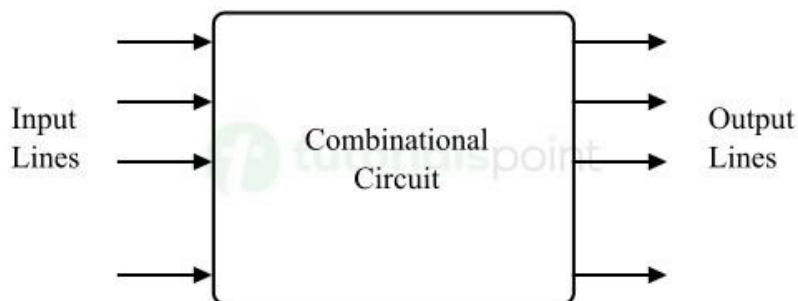
Unit-3

Short Question and Answers

1) **What is a Combinational logic circuit? Draw the Block diagram of combinational circuit.**

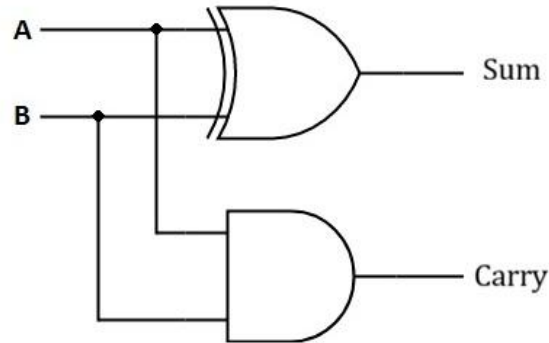
- **Ans:** A combinational circuit is a digital logic circuit whose output depends only on the present input values and does not depend on any feedback or previous input or output values.

Block Diagram of Combinational Circuit:



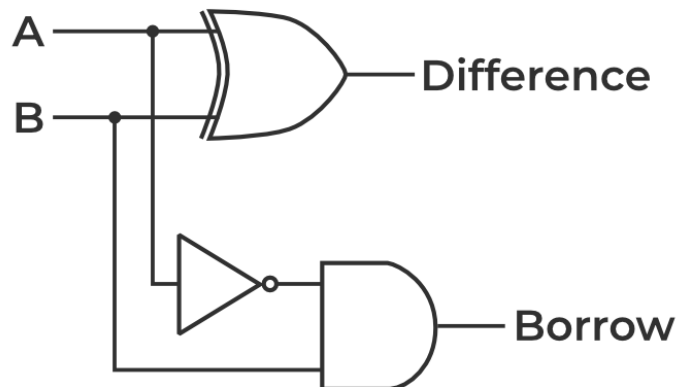


2) Draw the Half adder circuit diagram.



- The expression for sum S in half adder is: $S = A \oplus B$
- The expression for carry C in half adder is: $C = AB$

3) Draw the Half Binary subtractor circuit diagram.



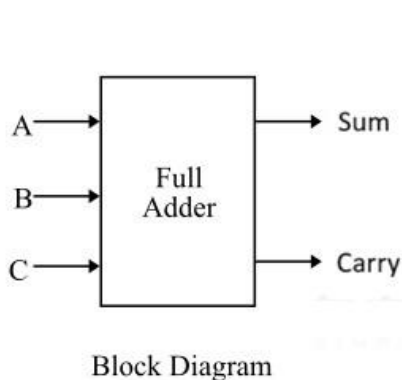
- The expression for Difference (D) in Half Binary subtractor is: $D = A \oplus B$
- The expression for Borrow (B) in Half Binary subtractor is: $B = \overline{A}.B$



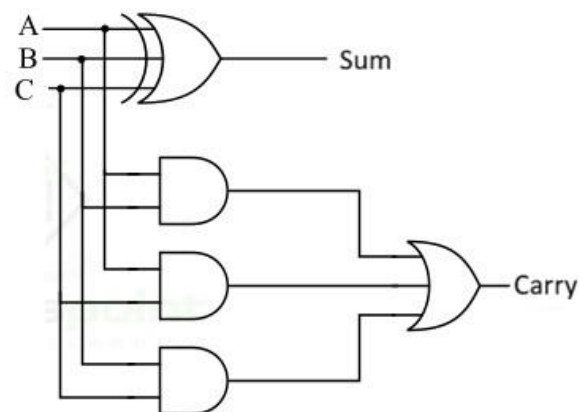
Long Question and Answers

1) Explain Full adder with logical circuit diagram and truth table.

- Full Adder is the circuit that consists of two EX-OR gates, two AND gates, and one OR gate.
- Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN.
- The output carry is designated as C-OUT and the normal output is designated as S which is SUM.
- The equation obtained by the EX-OR gate is the sum of the binary digits. While the output obtained by AND gate is the carry obtained by addition.



Block Diagram



Circuit Diagram

Inputs			Outputs	
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

truth table



K-Map for Full Adder:

K-Map (Karnaugh Map) is a method for simplifying binary complex Boolean algebraic expressions.

For Sum (S)

A \ BC _{in}	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$S = A'B'C_{in} + A'BC_{in} + AB'C_{in} + ABC_{in}$$
$$= A \oplus B \oplus C_{in}$$

For Output Carry (C_{out})

A \ BC _{in}	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$C_{out} = AB + AC_{in} + BC_{in}$$

- Sum = $(A \oplus B) \oplus C$
- Carry-out = $A.B + A.C + B.C$

2) Explain Half Binary Subtractor with circuit diagram and truth table.

- A Half Binary Subtractor is a combinational logic circuit that performs subtraction on two single binary digits (bits) producing Difference (D) and Borrow (B) bits.
- A Half Binary Subtractor can be constructed by combining an Exclusive-OR (XOR) gate and borrow bit generation.
- The resulting Truth Table of this Half Binary Subtractor along with the logic circuit is shown below:



Symbol	Truth Table			
	X	Y	DIFF.	BORR.
	0	0	0	0
	0	1	1	1
	1	0	1	0
	1	1	0	0

- It is obvious from the Truth Table and logic circuit that Difference (D) is the result of the Exclusive-OR (XOR) gate.
- Whereas, the Borrow (B) is the result of NOT-AND combinational logic.

K-Map for Half Binary subtractor:

For difference,

		A		
		0	1	
B	0	0	1	Difference(d)= $X \oplus Y$
	1	1	0	

For Borrow (B),

		B		
		0	1	
A	0	0	1	Borrow (B) = $\overline{X}.Y$
	1	0	0	

- The Boolean Expressions for these outputs are:

The expression for Difference (D) in Half Binary subtractor is: **$D = X \oplus Y$**

The expression for Borrow (B) in Half Binary subtractor is: **$B = \overline{X}.Y$**



3) Explain Full Binary Subtractor with circuit diagram and truth table.

- A Full Binary Subtractor has an additional input of “Borrow-in”, as compared to Half Binary Subtractor, for taking into account “Borrow-out” from the previous Half Subtractor.
- It contains three inputs(A, B, Bin) and produces two outputs (D, Bout).

Truth Table of Full Subtractor:

Inputs			Outputs	
A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-Map for Full Subtractor:

For Difference,

		B Bin			
		00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

$D = A'B'Bin + AB'Bin' + A'Bin' + ABBin$

The final simplified expression for the difference in a full subtractor is:

$$D = (A \oplus B) \oplus B_{in}$$



For Borrow (B) ,

		B Bin			
A		00	01	11	10
	0	0	1	1	1
	1	0	0	1	0

$Bout = A'Bin + A'B + BBin$

The final simplified expression for borrow in a full subtractor is:

$$\text{Borrow (B)} = BB_{in} + A'B + A'B_{in}$$

Logic Diagram of Full Subtractor:

